📂 CATEGORY 3: Bash Scripting — Complete Deep Dive

Linux & Bash Mastery — DevOps/SRE/Platform Engineer Interview Guide

---

Table of Contents

- 3.1 Shebang, Script Structure, Execution
- 3.2 Variables, Quoting Rules, export, env
- 3.3 Input/Output — echo, read, printf, Here-docs
- 3.4 Conditionals — if/elif/else, test, [[ ]] vs [ ]
- 3.5 Loops — for, while, until, break, continue
- 3.6 Functions — definition, local variables, return values, $?
- 3.7 Arrays & Associative Arrays
- 3.8 String Manipulation — Parameter Expansion
- 3.9 Exit Codes & Error Handling — set -e, set -o pipefail, trap
- 3.10 Process & Command Substitution — $(), <(), >()
- 3.11 Regex in Bash — =~, Character Classes, Anchors
- 3.12 xargs & Parallel Execution
- 3.13 File Descriptor Manipulation
- 3.14 Subshells vs Child Processes — () vs {}
- 3.15 Script Hardening — set -euo pipefail, Defensive Patterns
- 3.16 Performance in Bash — When to Use Bash vs Python
- Master Gotcha List
- Top Interview Questions

3.1 Shebang, Script Structure, Execution

🔷 What it is in simple terms

A Bash script is a plain text file containing shell commands executed sequentially. The **shebang** (`#!`) on line 1 tells the kernel **which interpreter** to use to run the file.

🔷 Why it exists / What problem it solves

Without a shebang, the kernel doesn't know if the file is a Bash script, Python script, Ruby script, or something else. The shebang makes scripts **self-describing** and **directly executable** — you run `./script.sh` instead of `bash script.sh`.

🔷 How it works internally

```
You type: ./deploy.sh
          │
          ▼
Kernel reads first 2 bytes: #!
          │
          ▼
Kernel reads interpreter path: /bin/bash
          │
          ▼
Kernel executes: /bin/bash ./deploy.sh
          │
          ▼
Bash reads script line by line
and executes commands
```

The kernel uses the `execve()` system call. When it detects `#!`, it reads the interpreter path and passes the script as an argument to that interpreter. This is handled entirely in kernel space — no shell involved yet.

🔷 Shebang variants — and which to use

```
#!/bin/bash          # Hardcoded bash path — works on most Linux systems
#!/usr/bin/env bash  # Finds bash in PATH — more portable (macOS, NixOS,
etc.)
#!/bin/sh            # POSIX sh — maximum portability, fewer features
#!/usr/bin/env python3  # Same pattern for Python scripts

# ⚠ Which to use?
# → For DevOps/SRE scripts: #!/usr/bin/env bash (most portable)
# → For scripts that must run anywhere: #!/bin/sh
# → Never: #!/bin/bash on macOS (bash is 3.x there, not 5.x)
```

🔷 Script structure — the anatomy of a production script

```
#!/usr/bin/env bash
#
==========================================================================
===
# Script: deploy.sh
# Description: Deploy application to production
# Author: SRE Team
# Date: 2024-01-15
# Usage: ./deploy.sh [environment] [version]
#
==========================================================================
===
```

```
# ── Error handling (always at the top) ──────────────────────────────
set -euo pipefail
IFS=$'\n\t'

# ── Constants ──────────────────────────────────────────────────────────
readonly SCRIPT_DIR="$(cd "$(dirname "${BASH_SOURCE[0]}")" && pwd)"
readonly SCRIPT_NAME="$(basename "$0")"
readonly LOG_FILE="/var/log/deploy.log"
readonly TIMESTAMP="$(date +%Y%m%d_%H%M%S)"

# ── Default values ─────────────────────────────────────────────────────
ENVIRONMENT="${1:-staging}"
VERSION="${2:-latest}"

# ── Functions ──────────────────────────────────────────────────────────
log() {
    echo "[$(date '+%Y-%m-%d %H:%M:%S')] $*" | tee -a "$LOG_FILE"
}

die() {
    log "ERROR: $*" >&2
    exit 1
}

usage() {
    cat <<EOF
Usage: $SCRIPT_NAME [environment] [version]

Arguments:
  environment   Target environment (default: staging)
  version       App version to deploy (default: latest)

Examples:
  $SCRIPT_NAME production v1.2.3
  $SCRIPT_NAME staging
EOF
    exit 0
}

# ── Validation ─────────────────────────────────────────────────────────
[[ "$ENVIRONMENT" =~ ^(staging|production)$ ]] || die "Invalid
environment: $ENVIRONMENT"
command -v docker >/dev/null 2>&1 || die "docker is required but not
installed"

# ── Main logic ─────────────────────────────────────────────────────────
main() {
    log "Starting deployment: env=$ENVIRONMENT version=$VERSION"
```

```
    # ... deployment logic ...
    log "Deployment complete"
}

# ── Entry point ──────────────────────────────────────────────────────
main "$@"
```

🔷 Execution methods — and critical differences

```
# Method 1: Direct execution (requires shebang + execute permission)
chmod +x script.sh
./script.sh              # Runs in a NEW child process

# Method 2: Explicit interpreter
bash script.sh           # No execute permission needed
sh script.sh             # Uses POSIX sh — may break bash-specific syntax

# Method 3: Source (dot command) — runs in CURRENT shell
source script.sh         # Changes affect current shell!
. script.sh              # Same as source

# Method 4: bash -x (debug mode)
bash -x script.sh        # Prints each command as executed (with + prefix)
bash -v script.sh        # Prints each line as read (before expansion)
bash -n script.sh        # Syntax check ONLY — does not execute

# Method 5: Set debug in-script
set -x    # Turn on debug tracing
set +x    # Turn off debug tracing

# Partial debug (trace only a section)
set -x
risky_command
set +x
```

🔷 `bash -x` — Your most important debugging tool

```
# Script:
#!/usr/bin/env bash
NAME="world"
echo "Hello $NAME"
ls /nonexistent

# bash -x output:
+ NAME=world                    # + prefix = command being executed
+ echo 'Hello world'            # Variables already expanded
```

```
Hello world
+ ls /nonexistent
ls: cannot access '/nonexistent': No such file or directory
```

```
# PS4 — customize the debug prefix (default is "+")
export PS4='+(${BASH_SOURCE}:${LINENO}): ${FUNCNAME[0]:+${FUNCNAME[0]}():
}'
bash -x script.sh
# +(script.sh:3): main(): NAME=world
# +(script.sh:4): main(): echo 'Hello world'
# Now you see filename + line number in every debug line!
```

🔷 `chmod +x` internals

```
chmod +x script.sh     # Adds execute bit for owner, group, others
chmod 755 script.sh    # rwxr-xr-x — typical for scripts

# What happens without execute bit:
./script.sh            # bash: ./script.sh: Permission denied
bash script.sh         # Still works! chmod not needed with explicit
interpreter

# Check permissions
ls -la script.sh
# -rwxr-xr-x 1 ubuntu ubuntu 1234 Jan 15 10:00 script.sh
#  ^^^           ^^^ execute bits set
```

🔷 `BASH_SOURCE` vs `$0`

```
# $0 = name of script as called
# BASH_SOURCE[0] = actual file path (even when sourced)

echo "$0"                    # ./script.sh  OR  bash  (if sourced!)
echo "${BASH_SOURCE[0]}"     # ./script.sh  (always the script file)

# Production pattern: get script's own directory reliably
SCRIPT_DIR="$(cd "$(dirname "${BASH_SOURCE[0]}")" && pwd)"
# Works even if called from another directory or via symlink
```

🔷 Short crisp interview answer

> "The shebang `#!/usr/bin/env bash` tells the kernel which interpreter to use. The kernel reads it via `execve()` and passes the script to bash. I prefer `env bash` over `/bin/bash` for portability. For execution, I use `chmod +x` for standalone scripts and `bash -x` for debugging — it prints every command with variables already expanded, making it indispensable for troubleshooting. Sourcing a script with `.` runs it in the current shell and can modify the current environment, unlike direct execution which creates a child process."

⚠ Tricky gotchas

```
# GOTCHA 1: Script runs differently based on execution method
export MY_VAR="hello"
./script.sh     # Child process — MY_VAR available only if exported
. script.sh     # Same process — ALL current shell vars available

# GOTCHA 2: Shebang path must be absolute
#!/usr/bin/env bash   # ✅ Correct
#! /bin/bash          # ✅ Space is allowed but unusual
bash                  # ❌ Not absolute — won't work as shebang

# GOTCHA 3: Windows line endings break shebangs
# If script has \r\n line endings:
./script.sh     # bash: ./script.sh: /bin/bash^M: bad interpreter
# Fix:
dos2unix script.sh
# Or:
sed -i 's/\r//' script.sh

# GOTCHA 4: .sh extension is NOT required
# Linux ignores extensions for execution — the shebang determines the
interpreter
# ./deploy (no extension) works perfectly fine

# GOTCHA 5: Current directory not in PATH
./script.sh    # ✅ Explicit path — works
script.sh      # ❌ Unless . is in PATH (security risk to add . to PATH!)
```

3.2 Variables, Quoting Rules, `export`, `env`

🔷 What it is

Variables in Bash store data — strings, numbers, arrays. Quoting controls how the shell **expands** and **interprets** that data. Getting quoting wrong is the #1 source of subtle bugs in shell scripts.

🔷 Variable basics

```
# Assignment — NO spaces around =
NAME="Alice"            # ✅
NAME = "Alice"          # ❌ Bash tries to run command named "NAME"

# Access with $
echo $NAME              # Alice (works but risky)
echo "$NAME"            # Alice (always double-quote!)
echo "${NAME}"          # Alice (explicit boundary — use when needed)

# Unset vs empty
unset NAME              # Variable doesn't exist
NAME=""                 # Variable exists but is empty
NAME=                   # Same as NAME=""

# Checking if set vs empty
[[ -v NAME ]]           # True if variable is SET (even if empty) — bash
4.2+
[[ -n "$NAME" ]]        # True if variable is non-empty
[[ -z "$NAME" ]]        # True if variable is empty or unset
```

🔷 Variable types

```
# String (default)
GREETING="Hello World"

# Integer (declare -i enforces integer type)
declare -i COUNT=0
COUNT="abc"             # Becomes 0 (non-numeric = 0 with declare -i)

# Readonly (constant)
readonly MAX_RETRIES=3
declare -r MAX_RETRIES=3  # Same thing
MAX_RETRIES=5           # bash: MAX_RETRIES: readonly variable

# Lowercase/uppercase (declare -l/-u)
declare -l LOWER="HELLO"   # Stores as "hello"
declare -u UPPER="hello"   # Stores as "HELLO"
```

🔷 Special variables — must know for interviews 🔥

```
$0          # Script name
$1-$9       # Positional parameters (arguments)
${10}       # 10th argument (need braces for 10+)
$#          # Number of arguments
$@          # All arguments as separate words  ← use this
$*          # All arguments as single string   ← avoid
```

```
$?          # Exit code of last command
$$          # PID of current shell
$!          # PID of last background process
$_          # Last argument of previous command
$-          # Current shell options (e.g., "himBH")
$LINENO     # Current line number in script
$RANDOM     # Random number 0-32767
$SECONDS    # Seconds since script started
$BASHPID    # PID of current bash process (differs from $$ in subshells)
```

🔷 Quoting rules — the most misunderstood topic ⚠️🔥

```
Three types of quoting:

┌──────────────────┬──────────────────────────────────────────┐
│ No quotes        │ Word splitting + glob expansion happen   │
│ "Double quotes"  │ Variable/command expansion, NO word splitting │
│ 'Single quotes'  │ No expansion whatsoever — literal string │
└──────────────────┴──────────────────────────────────────────┘
```

```bash
NAME="John Smith"
FILE="*.txt"

# No quotes — DANGEROUS with spaces/globs
echo $NAME         # Prints: John Smith (ok here)
ls $FILE           # Shell expands *.txt THEN passes to ls
cp $NAME /tmp/     # cp John Smith /tmp/ — cp gets 2 args! WRONG

# Double quotes — SAFE for variables
echo "$NAME"       # Prints: John Smith — preserved as one argument
ls "$FILE"         # ls "*.txt" — passes literal *.txt to ls (no glob!)
cp "$NAME" /tmp/   # cp "John Smith" /tmp/ — correct, one argument

# Single quotes — LITERAL everything
echo '$NAME'       # Prints: $NAME — no expansion
echo '*.txt'       # Prints: *.txt — no glob
echo 'It'\''s me'  # Prints: It's me (escape single quote outside quotes)

# Mixing quote types
echo "Hello '$NAME'"    # Hello 'John Smith' — vars expand, single quotes
literal
echo 'Hello "$NAME"'    # Hello "$NAME" — nothing expands
```

🔷 The critical $@ vs $* difference ⚠️

```
#!/usr/bin/env bash
show_args() {
    echo "Using \$@:"
    for arg in "$@"; do echo "  [$arg]"; done

    echo "Using \$*:"
    for arg in "$*"; do echo "  [$arg]"; done
}

show_args "hello world" "foo" "bar baz"

# Output:
# Using $@:
#   [hello world]    ← preserved as separate arg
#   [foo]
#   [bar baz]        ← preserved as separate arg
#
# Using $*:
#   [hello world foo bar baz]  ← ALL joined into ONE string!

# Rule: ALWAYS use "$@" when forwarding arguments
```

🔷 `export` — making variables available to child processes

```
# Without export — variable stays in current shell
NAME="Alice"
bash -c 'echo $NAME'   # Prints nothing — child doesn't see it

# With export — visible to all child processes
export NAME="Alice"
bash -c 'echo $NAME'   # Prints: Alice

# Export existing variable
NAME="Alice"
export NAME             # Now exported

# Export and assign in one line (most common)
export NAME="Alice"

# See all exported variables
env
export -p              # Same but with declare syntax

# Unexport (remove from environment but keep in shell)
export -n NAME
```

🔷 `env` command

```
# Print all environment variables
env

# Run command with CLEAN environment (no inherited vars)
env -i bash            # Completely clean shell
env -i PATH=/usr/bin:/bin ./script.sh  # Only PATH set

# Run command with extra/overridden env vars (without modifying shell)
env DB_HOST=newserver ./app
DATABASE_URL="postgres://..." env -S ./script.sh

# Check if variable is in environment
env | grep "^PATH="

# Production use: run script with different env
env -i HOME="$HOME" PATH="$PATH" USER="deploy" ./deploy.sh
```

🔷 Default values — parameter expansion 🔥

```
# Use default if unset or empty
echo "${NAME:-"Anonymous"}"     # "Anonymous" if NAME is unset/empty

# Use default and ASSIGN it
echo "${NAME:="Anonymous"}"     # Same + sets NAME variable

# Error if unset or empty
echo "${NAME:?"NAME is required"}"  # Exits with error if NAME is empty

# Use alternate value if SET
echo "${NAME:+"Name is set!"}"  # Prints "Name is set!" only if NAME is
set

# Without colon — only triggers on UNSET (not empty)
echo "${NAME-"Anonymous"}"      # "Anonymous" only if NAME is unset
                                # Empty NAME would print empty string
```

🔷 Short crisp interview answer

> "Bash variables are untyped strings by default. The critical rule is always double-quote variable expansions — `\"$VAR\"` — to prevent word splitting and glob expansion. `export` makes a variable available to child processes via the environment. `$@` and `$*` both hold all arguments but `\"$@\"` preserves each argument as separate words while `\"$*\"` joins them — so always use `\"$@\"` when forwarding args. Parameter expansion defaults like `${VAR:-default}` are essential for writing robust scripts that handle missing inputs gracefully."

⚠ Tricky gotchas

```
# GOTCHA 1: Spaces around = are syntax errors
NAME = "Alice"    # ERROR: tries to run command "NAME" with args

# GOTCHA 2: Subshell variable changes don't propagate up
(export FOO="bar")
echo $FOO         # Empty! Subshell can't modify parent

# GOTCHA 3: Arrays and word splitting
FILES="file1.txt file2.txt"
cp $FILES /backup/          # Works accidentally with simple names
cp "$FILES" /backup/        # WRONG — passes as one argument
FILES=(file1.txt file2.txt) # Use array instead
cp "${FILES[@]}" /backup/   # CORRECT

# GOTCHA 4: Unbound variable trap with set -u
set -u
echo $UNDEFINED_VAR   # bash: UNDEFINED_VAR: unbound variable — script
exits!
echo "${UNDEFINED_VAR:-}"  # Safe — provides empty default

# GOTCHA 5: env vs export
export FOO=bar    # Sets in current shell AND environment
env FOO=bar cmd   # Sets for 'cmd' ONLY — current shell unchanged
```

3.3 Input/Output — `echo`, `read`, `printf`, Here-docs

🔷 `echo` vs `printf` — know the difference ⚠

```
# echo — simple, but inconsistent across systems
echo "Hello World"
echo -n "No newline"      # -n suppresses newline
echo -e "Tab:\there"      # -e enables escape sequences
# ⚠ echo -e behavior differs between bash built-in and /bin/echo
# ⚠ echo -n is not POSIX

# printf — consistent, portable, powerful (ALWAYS prefer in scripts)
printf "Hello World\n"              # Always explicit about newline
printf "Name: %s, Age: %d\n" "Alice" 30
printf "%-20s %5d\n" "item" 42      # Left-align, right-align with padding
printf "%05d\n" 42                  # Zero-pad: 00042
printf "%.2f\n" 3.14159             # 2 decimal places: 3.14
printf "%x\n" 255                   # Hex: ff
printf "%b\n" "Hello\tWorld"        # Process escape sequences

# printf to variable (no subshell!)
printf -v RESULT "%.2f" 3.14159     # Stores into $RESULT — efficient
```

🔷 `read` — Reading input

```
# Basic read
read NAME                   # Reads line, stores in NAME
echo "Hello, $NAME"

# Read with prompt
read -p "Enter name: " NAME

# Read silently (passwords!)
read -s -p "Password: " PASS
echo                        # Print newline after silent read

# Read with timeout
read -t 10 -p "Enter (10s timeout): " INPUT || echo "Timeout!"

# Read into array (each word becomes element)
read -a SERVERS <<< "web01 web02 db01"
echo "${SERVERS[0]}"        # web01

# Read with delimiter
read -d ':' FIELD           # Read until : instead of newline

# Read multiple variables at once
read NAME AGE ROLE <<< "Alice 30 Engineer"
echo "$NAME $AGE $ROLE"     # Alice 30 Engineer

# Read line by line from file
while IFS= read -r line; do
    echo "Line: $line"
done < /etc/hosts
# IFS= prevents stripping leading/trailing whitespace
# -r prevents backslash interpretation

# Read from command output
while IFS= read -r line; do
    echo "Processing: $line"
done < <(find /var/log -name "*.log")
```

🔷 Here-documents (heredoc)

```
# Basic heredoc — send multiline text to command
cat << 'EOF'
This is line 1
This is line 2
Variables like $HOME are NOT expanded (single-quoted delimiter)
EOF
```

```
# With variable expansion (unquoted delimiter)
NAME="Alice"
cat << EOF
Hello, $NAME!
Today is $(date)
Your home is $HOME
EOF

# Indented heredoc (bash 4.4+, strips leading tabs)
cat <<- EOF
    This line has a leading tab — it gets stripped
    So does this one
    EOF

# Heredoc into variable
SQL=$(cat << EOF
SELECT *
FROM users
WHERE active = 1
  AND created_at > '2024-01-01'
ORDER BY name;
EOF
)
echo "$SQL"

# Heredoc to file
cat > /etc/app/config.yaml << EOF
database:
  host: ${DB_HOST}
  port: ${DB_PORT:-5432}
  name: ${DB_NAME}
EOF

# Heredoc to multiple commands on remote server
ssh user@server << 'EOF'
  cd /opt/app
  git pull origin main
  systemctl restart app
EOF

# Herestring (single line)
grep "pattern" <<< "this is the string to search"
wc -w <<< "count these words"
```

🔷 Redirection — the full picture

```
# Standard streams
# stdin  = fd 0 (keyboard by default)
# stdout = fd 1 (terminal by default)
```

```
# stderr = fd 2 (terminal by default)

command > file          # Redirect stdout to file (overwrite)
command >> file         # Redirect stdout to file (append)
command 2> file         # Redirect stderr to file
command 2>&1            # Redirect stderr TO stdout (merge them)
command > file 2>&1     # Both stdout and stderr to file
command &> file         # Shorthand for above (bash only)
command 2>/dev/null     # Discard stderr
command >/dev/null 2>&1 # Discard ALL output

# ⚠ Order matters with redirections!
command > file 2>&1     # ✅ stderr goes to where stdout points (file)
command 2>&1 > file     # ❌ stderr goes to original stdout (terminal)

# Input redirection
command < file          # Read stdin from file
command <<< "string"    # Herestring — string as stdin

# Pipelines
cmd1 | cmd2             # stdout of cmd1 → stdin of cmd2
cmd1 |& cmd2            # stdout AND stderr of cmd1 → stdin of cmd2 (bash
4+)

# tee — write to file AND pass through
command | tee file.log         # stdout → file AND stdout
command | tee -a file.log      # Append mode
command | tee file.log | grep "ERROR"  # Chain further
```

🔷 Short crisp interview answer

> "For output, I prefer `printf` over `echo` in scripts because it's consistent across systems and allows format strings. For input, `read -r` is the safe form — the `-r` prevents backslash interpretation. For multiline input, heredocs with `<< 'EOF'` (single-quoted) prevent variable expansion when you want literal text. The most critical redirection to know is `2>&1` to merge stderr into stdout — but order matters: `> file 2>&1` is correct, `2>&1 > file` doesn't do what you expect."

⚠ Tricky gotchas

```
# GOTCHA 1: read in a pipeline runs in a subshell — variable lost!
echo "hello" | read VAR
echo $VAR     # EMPTY! read ran in subshell

# Fix: use herestring
read VAR <<< "hello"
echo $VAR     # hello

# GOTCHA 2: IFS and read
IFS=: read -r USER PASS UID GID <<< "root:x:0:0"
```

```
# Splits on : and assigns to each variable

# GOTCHA 3: echo -e is not portable
echo -e "hello\tworld"        # Works in bash built-in
/bin/echo -e "hello\tworld"   # May not work on all systems
printf "hello\tworld\n"       # Always works

# GOTCHA 4: Heredoc indentation
cat << EOF
    indented
EOF
# The spaces ARE in the output — use <<- with TABS (not spaces) to strip
```

3.4 Conditionals — `if/elif/else`, `test`, `[[ ]]` vs `[ ]`

🔷 How `if` really works

```
# if doesn't test conditions — it runs a COMMAND and checks its EXIT CODE
# Exit code 0 = true (success)
# Exit code non-0 = false (failure)

if COMMAND; then
    # runs if COMMAND exited with 0
fi

# These are all equivalent:
if [ "$a" = "$b" ]; then ...
if test "$a" = "$b"; then ...
# Because [ is literally a command (synonym for test)!

ls /tmp            # exit 0 = exists
if ls /tmp; then echo "exists"; fi   # Totally valid!
```

🔷 `[ ]` vs `[[ ]]` — the critical difference ⚠️🔥

```
┌──────────────────────┬──────────────────────────────────────┐
│ [ ]  (test)          │ [[ ]] (bash keyword)                 │
├──────────────────────┼──────────────────────────────────────┤
│ POSIX — works in sh  │ Bash-only                            │
│ Word splitting       │ No word splitting — safer with variables │
│ Glob expansion       │ No glob expansion                    │
│ No regex             │ Supports =~ regex matching           │
│ Need to quote vars   │ Quoting optional (but still good practice) │
│ && → -a, || → -o     │ Use && and || directly               │
│ < > need escaping    │ < > work directly (string comparison) │
└──────────────────────┴──────────────────────────────────────┘
```

```
# [ ] pitfalls
FILE="my file.txt"
[ -f $FILE ]       # ❌ Word splits to: [ -f my file.txt ] — error!
[ -f "$FILE" ]     # ✅ Quoted

# [[ ]] is safer
[[ -f $FILE ]]     # ✅ No word splitting even without quotes

# Regex — only in [[ ]]
[[ "hello123" =~ ^[a-z]+[0-9]+$ ]] && echo "matches"

# AND/OR
[ -f file ] && [ -r file ]   # [ ] style
[[ -f file && -r file ]]     # [[ ]] style — cleaner

# String comparison with <
[ "apple" \< "banana" ]      # ❌ Must escape < in [ ]
[[ "apple" < "banana" ]]     # ✅ Direct in [[ ]]
```

🔷 Test operators — complete reference

```
# ── File tests ──────────────────────────────────────────────
-e file      # exists (any type)
-f file      # exists and is regular file
-d file      # exists and is directory
-l file      # exists and is symbolic link
-r file      # readable
-w file      # writable
-x file      # executable
-s file      # exists and size > 0 (non-empty)
-b file      # block device
-c file      # character device
-p file      # named pipe (FIFO)
-S file      # socket
file1 -nt file2  # file1 newer than file2
file1 -ot file2  # file1 older than file2
file1 -ef file2  # same file (same inode)

# ── String tests ────────────────────────────────────────────
-z "$str"    # zero length (empty)
-n "$str"    # non-zero length (non-empty)
"$a" = "$b"  # equal (use = in [ ], == works in [[ ]])
"$a" != "$b" # not equal
"$a" < "$b"  # less than (lexicographic)
"$a" > "$b"  # greater than

# ── Integer tests ───────────────────────────────────────────
$a -eq $b    # equal
```

```
$a -ne $b    # not equal
$a -lt $b    # less than
$a -le $b    # less than or equal
$a -gt $b    # greater than
$a -ge $b    # greater than or equal

# ── Logic ──────────────────────────────────────────────────────────
! expr          # NOT
expr1 -a expr2  # AND (in [ ])
expr1 -o expr2  # OR (in [ ])
```

🔷 Full conditional examples

```
# Basic if/elif/else
if [[ "$ENV" == "production" ]]; then
    echo "Production deployment"
elif [[ "$ENV" == "staging" ]]; then
    echo "Staging deployment"
else
    echo "Unknown environment: $ENV"
    exit 1
fi

# Null check pattern
if [[ -z "${API_KEY:-}" ]]; then
    echo "ERROR: API_KEY is required" >&2
    exit 1
fi

# File existence check
if [[ ! -f "/etc/app/config.yaml" ]]; then
    echo "Config file missing!" >&2
    exit 1
fi

# Check command exists
if ! command -v jq &>/dev/null; then
    echo "jq is required. Install with: apt install jq"
    exit 1
fi

# Compound conditions
if [[ -f "$CONFIG" && -r "$CONFIG" && -s "$CONFIG" ]]; then
    source "$CONFIG"
fi

# One-liner conditionals (common in scripts)
[[ -d /tmp/work ]] || mkdir -p /tmp/work
[[ -n "$DEBUG" ]] && set -x
command -v curl &>/dev/null || { echo "curl required"; exit 1; }
```

```
# Case statement (cleaner than long if/elif)
case "$ENVIRONMENT" in
    production|prod)
        DB_HOST="prod-db.internal"
        REPLICAS=5
        ;;
    staging|stg)
        DB_HOST="staging-db.internal"
        REPLICAS=2
        ;;
    local|dev)
        DB_HOST="localhost"
        REPLICAS=1
        ;;
    *)
        echo "Unknown environment: $ENVIRONMENT" >&2
        exit 1
        ;;
esac

# Arithmetic conditionals
COUNT=5
if (( COUNT > 3 )); then      # (( )) for arithmetic — no $ needed
    echo "Count exceeds limit"
fi
(( COUNT++ ))                  # Increment
(( TOTAL = A + B ))            # Arithmetic assignment
```

🔷 Short crisp interview answer

> "`if` in Bash runs a command and checks its exit code — zero is true, non-zero is false. `[[ ]]` is the preferred form for conditionals in modern Bash — it's a keyword, not a command, so it doesn't suffer from word splitting, supports regex with `=~`, and lets you use `&&` and `||` directly. `[ ]` is POSIX-compatible but requires careful quoting and escaping. For arithmetic, I use `(( ))` which lets me write conditions like `(( count > 5 ))` without dollar signs or comparison flags."

⚠ Tricky gotchas

```
# GOTCHA 1: = vs == in [ ]
[ "$a" = "$b" ]    # ✅ POSIX — use = for string comparison in [ ]
[ "$a" == "$b" ]   # Works in bash [ ] but not POSIX sh

# GOTCHA 2: Integer vs string comparison
[ "10" -gt "9" ]   # ✅ Integer: 10 > 9
[ "10" > "9" ]     # ❌ String: "10" < "9" lexicographically!

# GOTCHA 3: Unquoted variable in [ ] with empty value
VAR=""
```

```
[ -n $VAR ]         # [ -n ] — too few arguments! ERROR
[ -n "$VAR" ]       # ✅ Correct

# GOTCHA 4: Spaces inside [ ] are mandatory
["$a" = "$b"]       # ❌ Command not found: [hello
[ "$a" = "$b" ]     # ✅ Spaces required around [ and ]

# GOTCHA 5: && and || precedence outside [[
[ -f file ] || echo "missing" && exit 1
# This is: ([ -f file ] || echo "missing") && exit 1 — WRONG!
# Always use braces:
[ -f file ] || { echo "missing"; exit 1; }
```

3.5 Loops — `for`, `while`, `until`, `break`, `continue`

🔷 `for` loop — three forms

```
# Form 1: List iteration (most common)
for SERVER in web01 web02 web03 db01; do
    echo "Pinging $SERVER..."
    ping -c 1 "$SERVER" &>/dev/null && echo "$SERVER: UP" || echo
"$SERVER: DOWN"
done

# Form 2: C-style arithmetic loop
for (( i=0; i<10; i++ )); do
    echo "Iteration: $i"
done

# Form 3: Range with brace expansion
for i in {1..10}; do
    echo "Item $i"
done

for i in {0..100..10}; do   # Step by 10: 0 10 20 ... 100
    echo "$i"
done

# Iterate over array
SERVERS=(web01 web02 db01)
for server in "${SERVERS[@]}"; do
    ssh "$server" "systemctl status nginx"
done

# Iterate over files
for file in /var/log/*.log; do
    [[ -f "$file" ]] || continue    # Skip if no .log files match
    echo "Processing: $file"
    gzip "$file"
done
```

```bash
# Iterate over lines of a file — SAFE pattern
while IFS= read -r line; do
    echo "$line"
done < /etc/hosts

# Iterate over command output
for pid in $(pgrep nginx); do
    echo "nginx PID: $pid"
    cat /proc/"$pid"/status | grep VmRSS
done
```

🔷 `while` loop

```bash
# Basic while
COUNT=0
while [[ $COUNT -lt 5 ]]; do
    echo "Count: $COUNT"
    (( COUNT++ ))
done

# While with read — THE standard pattern for file/stream processing
while IFS= read -r line; do
    echo "Processing: $line"
done < /etc/passwd

# While reading command output
while IFS= read -r line; do
    echo "Log: $line"
done < <(journalctl -u nginx --since "1 hour ago")

# Infinite loop with break condition
while true; do
    if check_service_health; then
        echo "Service healthy"
        break
    fi
    echo "Waiting for service..."
    sleep 5
done

# Retry pattern — common in SRE scripts
MAX_RETRIES=5
RETRY_COUNT=0
while ! curl -sf "https://api.example.com/health"; do
    RETRY_COUNT=$(( RETRY_COUNT + 1 ))
    if [[ $RETRY_COUNT -ge $MAX_RETRIES ]]; then
        echo "ERROR: Service failed after $MAX_RETRIES attempts" >&2
        exit 1
    fi
```

```
    echo "Attempt $RETRY_COUNT failed, retrying in 10s..."
    sleep 10
done
```

🔷 `until` loop

```
# until — runs WHILE condition is FALSE (opposite of while)
RETRIES=0
until ping -c1 google.com &>/dev/null; do
    echo "Network not ready, waiting..."
    sleep 2
    (( RETRIES++ ))
    [[ $RETRIES -ge 10 ]] && { echo "Network timeout"; exit 1; }
done
echo "Network is up!"
```

🔷 `break` and `continue`

```
# break — exit loop immediately
for i in {1..100}; do
    [[ $i -eq 50 ]] && break
    echo $i
done

# break N — break out of N nested loops
for i in {1..3}; do
    for j in {1..3}; do
        [[ $i -eq 2 && $j -eq 2 ]] && break 2   # Breaks BOTH loops
        echo "$i $j"
    done
done

# continue — skip to next iteration
for i in {1..10}; do
    (( i % 2 == 0 )) && continue    # Skip even numbers
    echo "Odd: $i"
done

# continue N — continue outer loop from inner loop
for server in "${SERVERS[@]}"; do
    for port in 80 443 8080; do
        if ! nc -z "$server" "$port" 2>/dev/null; then
            echo "$server:$port CLOSED"
            continue 2    # Skip to next server
        fi
    done
```

```
    echo "$server: all ports open"
done
```

🔷 Real production loop examples

```
# Deploy to multiple servers in parallel
SERVERS=(web01 web02 web03 web04)
PIDS=()
for server in "${SERVERS[@]}"; do
    ssh "$server" "./deploy.sh" &
    PIDS+=($!)
done

# Wait for all and check exit codes
for pid in "${PIDS[@]}"; do
    if ! wait "$pid"; then
        echo "ERROR: deployment failed for PID $pid" >&2
    fi
done

# Process log files older than 30 days
find /var/log/app -name "*.log" -mtime +30 | while IFS= read -r file; do
    gzip "$file" && echo "Compressed: $file"
done

# Batch API calls with rate limiting
while IFS=, read -r id name email; do
    curl -s -X POST "https://api.example.com/users" \
         -H "Content-Type: application/json" \
         -d "{\"id\":\"$id\",\"name\":\"$name\",\"email\":\"$email\"}"
    sleep 0.1    # Rate limit: 10 requests/second
done < users.csv
```

🔷 Short crisp interview answer

> "Bash has three loop types: `for` for iterating over lists or ranges, `while` for condition-based looping, and `until` which is the inverse of while. The most important production pattern is `while IFS= read -r line; do ... done < file` for safe line-by-line file processing — the `IFS=` prevents whitespace stripping and `-r` prevents backslash interpretation. I use `break N` and `continue N` to control nested loops. One critical gotcha is that variables modified inside a while loop reading from a pipe are lost afterward because the pipe creates a subshell."

⚠ Tricky gotchas

```
# GOTCHA 1: for loop on glob that matches nothing
for f in /tmp/*.xyz; do
    echo "$f"    # Prints "/tmp/*.xyz" literally if no files match!
done
# Fix:
shopt -s nullglob          # Makes unmatched globs expand to nothing
for f in /tmp/*.xyz; do
    [[ -f "$f" ]] || continue
    echo "$f"
done

# GOTCHA 2: While loop in pipe = subshell — variables lost!
TOTAL=0
cat numbers.txt | while read -r n; do
    (( TOTAL += n ))
done
echo "Total: $TOTAL"    # Always 0! TOTAL modified in subshell

# Fix: use process substitution
while read -r n; do
    (( TOTAL += n ))
done < <(cat numbers.txt)
echo "Total: $TOTAL"    # Correct!

# GOTCHA 3: for with command substitution and newlines
for file in $(find /tmp -name "*.txt"); do
    echo "$file"
done
# Breaks if filenames have spaces! Use while read instead:
while IFS= read -r file; do
    echo "$file"
done < <(find /tmp -name "*.txt")

# GOTCHA 4: Modifying loop variable doesn't affect loop
for i in 1 2 3; do
    i=99    # Doesn't affect loop — next iteration still uses list
done
```

3.6 Functions — Definition, Local Variables, Return Values, `$?`

🔷 What it is

Functions let you group commands into a reusable named block. In Bash, functions are first-class — they appear in the environment, can be exported, and behave like mini-scripts.

🔷 Function definition syntax

```
# Style 1: function keyword
function greet() {
    echo "Hello, $1"
}

# Style 2: POSIX style (preferred — more portable)
greet() {
    echo "Hello, $1"
}

# Both are equivalent in bash. Use Style 2 for portability.
```

🔷 Arguments and return values ⚠️

```
# Functions receive arguments as $1, $2, etc. (same as scripts)
# BUT $0 is still the script name, not function name
# Use $FUNCNAME[0] for function name

deploy() {
    local env="$1"
    local version="$2"
    echo "Deploying $version to $env"
    echo "Function name: ${FUNCNAME[0]}"  # deploy
    echo "Script name: $0"               # ./script.sh
}

deploy production v1.2.3

# ── Return values ──────────────────────────────────────────────
# Bash functions can only RETURN an exit code (0-255) via 'return'
# For actual data, use: stdout capture, global variables, or nameref

# Method 1: Return exit code only (boolean pass/fail)
is_even() {
    (( $1 % 2 == 0 ))   # Returns 0 if even, 1 if odd
}
if is_even 4; then echo "even"; fi

# Method 2: Echo the result (capture with $())
get_timestamp() {
    date +%Y%m%d_%H%M%S
}
TIMESTAMP=$(get_timestamp)    # Captures stdout — runs in subshell!
echo "Timestamp: $TIMESTAMP"

# Method 3: Write to a named variable (no subshell) — bash 4.3+
get_server_count() {
    local -n result_ref=$1    # nameref
```

```
    result_ref=$(wc -l < /etc/hosts)
}
get_server_count MY_COUNT
echo "Server count: $MY_COUNT"

# Method 4: Global variable (simple but messy at scale)
RESULT=""
calculate() {
    RESULT=$(( $1 + $2 ))
}
calculate 3 4
echo "Result: $RESULT"
```

🔷 `local` variables — critical for scope

```
# Without local — variable leaks into global scope!
bad_function() {
    COUNTER=0              # GLOBAL — modifies outer scope!
    COUNTER=$(( COUNTER + 1 ))
}

# With local — variable is scoped to function
good_function() {
    local counter=0        # LOCAL — doesn't affect outer scope
    local -i count=0       # Local integer
    local -a items=()      # Local array
    local -r CONST="fixed" # Local readonly
    counter=$(( counter + 1 ))
}

# Scope demonstration
COUNTER=100
bad_function
echo $COUNTER    # 1 — MODIFIED by function!

COUNTER=100
good_function
echo $COUNTER    # 100 — UNCHANGED
```

🔷 `$?` — Exit code handling

```
# $? holds exit code of LAST command
ls /tmp
echo $?          # 0 (success)

ls /nonexistent
echo $?          # 2 (error)
```

```
# ⚠ $? is reset by every command — capture immediately!
some_command
exit_code=$?     # Capture IMMEDIATELY
echo "doing something else"
echo $?          # Now $? is echo's exit code, not some_command's!
echo "Exit was: $exit_code"  # Correct

# Function exit codes
check_disk() {
    local usage
    usage=$(df / | awk 'NR==2 {print $5}' | tr -d '%')
    if (( usage > 90 )); then
        echo "CRITICAL: Disk usage at ${usage}%" >&2
        return 2
    elif (( usage > 75 )); then
        echo "WARNING: Disk usage at ${usage}%" >&2
        return 1
    fi
    echo "OK: Disk usage at ${usage}%"
    return 0
}

check_disk
case $? in
    0) echo "Disk OK" ;;
    1) echo "Disk warning — monitor closely" ;;
    2) echo "Disk critical — take action!" ;;
esac
```

🔷 Advanced function patterns

```
# Function library pattern — source from scripts
# lib/logging.sh
LOG_LEVEL="${LOG_LEVEL:-INFO}"

log_info()  { echo "[INFO]  $(date '+%H:%M:%S') $*"; }
log_warn()  { echo "[WARN]  $(date '+%H:%M:%S') $*" >&2; }
log_error() { echo "[ERROR] $(date '+%H:%M:%S') $*" >&2; }
log_debug() { [[ "$LOG_LEVEL" == "DEBUG" ]] && echo "[DEBUG] $(date
'+%H:%M:%S') $*"; }

# In main script:
source "$(dirname "${BASH_SOURCE[0]}")/lib/logging.sh"

# Export function to subshells
greet() { echo "Hello, $1"; }
export -f greet           # Now available in child bash processes
bash -c 'greet World'     # Works!
```

```bash
# Function with cleanup trap
with_tempdir() {
    local tmpdir
    tmpdir=$(mktemp -d)
    trap "rm -rf '$tmpdir'" RETURN    # Cleanup on function return
    cd "$tmpdir"
    "$@"    # Run arguments as command in tempdir
}

# Check if function exists
if declare -f my_function &>/dev/null; then
    my_function
fi
```

🔷 Short crisp interview answer

> "Functions in Bash group commands into reusable blocks. Critical rules: always use `local` for variables to prevent scope leakage, functions can only `return` exit codes 0-255, and for actual data output I either capture stdout via `$()` or write to a variable by reference using bash 4.3+ namerefs. `$?` captures the last exit code but must be saved immediately because the next command resets it. I export functions with `export -f` when they need to be available in subshells like `xargs` or parallel execution."

⚠ Tricky gotchas

```bash
# GOTCHA 1: return vs exit in functions
exit_me() {
    exit 1    # This exits the ENTIRE SCRIPT, not just the function!
}
return_me() {
    return 1  # This exits only the function — correct
}

# GOTCHA 2: $() captures stdout — send debug to stderr!
debug_func() {
    echo "DEBUG: starting" >&2    # Send debug to stderr, not stdout
    echo "actual result"           # This is the return value
}
RESULT=$(debug_func)   # RESULT="actual result", debug goes to terminal

# GOTCHA 3: Function not defined before call
my_script() {
    helper    # ❌ Error if helper is defined AFTER my_script
}
# Fix: define helpers before callers, or use main() pattern

# GOTCHA 4: Subshell in $() means no variable side effects
populate_array() {
    MY_ARRAY=(one two three)
```

```
}
RESULT=$(populate_array)   # Runs in subshell — MY_ARRAY is lost!
populate_array             # Call directly if you need side effects
```

3.7 Arrays & Associative Arrays

🔷 Indexed arrays

```
# Declaration
declare -a FRUITS           # Explicit (optional)
FRUITS=("apple" "banana" "cherry")

# Access
echo "${FRUITS[0]}"         # apple (zero-indexed)
echo "${FRUITS[-1]}"        # cherry (negative index from bash 4.3)
echo "${FRUITS[@]}"         # All elements: apple banana cherry
echo "${#FRUITS[@]}"        # Length: 3
echo "${!FRUITS[@]}"        # Indices: 0 1 2

# Append
FRUITS+=("date" "elderberry")
FRUITS[10]="fig"            # Sparse array — indices 5-9 don't exist

# Slice
echo "${FRUITS[@]:1:2}"     # Elements 1-2: banana cherry
echo "${FRUITS[@]:1}"       # From index 1 onwards

# Delete element
unset 'FRUITS[1]'           # Removes banana, array is now sparse

# Iterate safely
for fruit in "${FRUITS[@]}"; do
    echo "$fruit"
done

# Iterate with index
for i in "${!FRUITS[@]}"; do
    echo "Index $i: ${FRUITS[$i]}"
done

# Build array from command output
PIDS=( $(pgrep nginx) )

# Build array from file lines — bash 4.0+
mapfile -t LINES < /etc/hosts      # preferred
readarray -t LINES < /etc/hosts    # Same as mapfile

# Convert string to array
IFS=',' read -ra TAGS <<< "web,api,db,cache"
echo "${TAGS[0]}"   # web
```

🔷 Associative arrays (hash maps) — bash 4.0+

```
# Must explicitly declare
declare -A SERVER_IPS
SERVER_IPS["web01"]="10.0.0.1"
SERVER_IPS["web02"]="10.0.0.2"
SERVER_IPS["db01"]="10.0.0.3"

# Or initialize inline
declare -A CONFIG=(
    ["env"]="production"
    ["region"]="us-east-1"
    ["replicas"]="3"
)

# Access
echo "${SERVER_IPS["web01"]}"     # 10.0.0.1
echo "${CONFIG["region"]}"        # us-east-1

# Check if key exists
if [[ -v SERVER_IPS["web01"] ]]; then
    echo "web01 is registered"
fi

# All keys
echo "${!SERVER_IPS[@]}"         # web01 web02 db01

# All values
echo "${SERVER_IPS[@]}"          # 10.0.0.1 10.0.0.2 10.0.0.3

# Length (number of key-value pairs)
echo "${#SERVER_IPS[@]}"         # 3

# Iterate
for host in "${!SERVER_IPS[@]}"; do
    echo "$host → ${SERVER_IPS[$host]}"
done

# Delete key
unset 'SERVER_IPS["web02"]'

# Real-world: config map pattern
declare -A REPLICA_COUNT=(
    ["production"]=5
    ["staging"]=2
    ["dev"]=1
)
REPLICAS="${REPLICA_COUNT["${ENVIRONMENT:-dev}"]}"
echo "Starting $REPLICAS replicas"
```

🔷 Short crisp interview answer

> "Bash supports two array types: indexed arrays (like lists, zero-based) and associative arrays (like hash maps, requires `declare -A`). The critical syntax is `\"${ARRAY[@]}\"` with double quotes to preserve elements with spaces. `${#ARRAY[@]}` gives the count, `${!ARRAY[@]}` gives the indices/keys. For safe line-by-line population I use `mapfile -t ARRAY < file`. Associative arrays are extremely useful for config maps and frequency counting in scripts."

⚠️ Tricky gotchas

```
# GOTCHA 1: $ARRAY is NOT the array — it's just element [0]
echo $FRUITS         # Only prints first element!
echo "${FRUITS[@]}"  # All elements

# GOTCHA 2: Quoting for elements with spaces
FILES=("my file.txt" "another file.txt")
for f in ${FILES[@]}; do   # ❌ Word splits! Gets 4 items, not 2
for f in "${FILES[@]}"; do # ✅ Correct — 2 items

# GOTCHA 3: declare -A requires bash 4.0+
# macOS default bash is 3.2 — no associative arrays!
# Check: bash --version

# GOTCHA 4: Unset creates sparse array
NUMS=(1 2 3 4 5)
unset 'NUMS[2]'
echo "${#NUMS[@]}"   # 4 (correct count)
echo "${NUMS[2]}"    # Empty (element gone but index gap exists)
for n in "${NUMS[@]}"; do echo $n; done   # Skips the gap safely
```

3.8 String Manipulation — Parameter Expansion

🔷 The complete parameter expansion toolkit

```
VAR="Hello, World! Hello, Bash!"

# ── Length ──────────────────────────────────────────────────────
echo "${#VAR}"                    # 28

# ── Substring extraction ────────────────────────────────────────
echo "${VAR:7}"                   # World! Hello, Bash!
echo "${VAR:7:5}"                 # World
echo "${VAR: -5}"                 # Bash! (negative = from end)
echo "${VAR: -5:4}"               # Bash
```

```
# ── Pattern removal (prefix) ──────────────────────────────────────
FILE="/var/log/app/error.log"
echo "${FILE#*/}"                 # var/log/app/error.log (shortest
prefix)
echo "${FILE##*/}"                # error.log             (longest prefix
= basename!)

# ── Pattern removal (suffix) ──────────────────────────────────────
echo "${FILE%/*}"                 # /var/log/app          (shortest suffix
= dirname!)
echo "${FILE%%/*}"                # (empty)
echo "${FILE%.log}"               # /var/log/app/error    (remove
extension)

# ── Substitution ──────────────────────────────────────────────────
echo "${VAR/Hello/Hi}"            # Hi, World! Hello, Bash!   (first
match)
echo "${VAR//Hello/Hi}"           # Hi, World! Hi, Bash!      (all
matches)
echo "${VAR/#Hello/Hi}"           # Hi, World! Hello, Bash!   (prefix
only)
echo "${VAR/%Bash!/Goodbye!}"     # Hello, World! Hello, Goodbye! (suffix
only)

# ── Case conversion (bash 4.0+) ───────────────────────────────────
STR="Hello World"
echo "${STR,,}"                   # hello world (all lowercase)
echo "${STR^^}"                   # HELLO WORLD (all uppercase)
echo "${STR,}"                    # hELLO WORLD (first char lowercase)
echo "${STR^}"                    # Hello World (first char uppercase)

# ── Filename manipulation patterns ────────────────────────────────
PATH_VAR="/var/log/nginx/access.log.gz"
echo "${PATH_VAR##*/}"            # access.log.gz       (filename)
echo "${PATH_VAR%/*}"             # /var/log/nginx      (directory)
echo "${PATH_VAR##*.}"            # gz                  (extension)
echo "${PATH_VAR%.*}"             # /var/log/nginx/access.log
echo "${PATH_VAR%%.*}"            # /var/log/nginx/access (remove all
exts)

# ── Real-world: rename files
for file in *.txt; do
    mv "$file" "${file%.txt}.md"       # Change .txt to .md
done

for file in *.JPG; do
    mv "$file" "${file%.JPG}.jpg"      # JPG to jpg
done
```

🔷 String tests and operations

```
# Check if string contains substring
if [[ "$URL" == *"amazonaws.com"* ]]; then
    echo "AWS URL"
fi

# Check prefix/suffix
if [[ "$FILE" == *.log ]]; then echo "Log file"; fi
if [[ "$URL" == https://* ]]; then echo "Secure URL"; fi

# Regex match — extract group
if [[ "$VERSION" =~ ^v([0-9]+)\.([0-9]+)\.([0-9]+)$ ]]; then
    MAJOR="${BASH_REMATCH[1]}"
    MINOR="${BASH_REMATCH[2]}"
    PATCH="${BASH_REMATCH[3]}"
    echo "Major: $MAJOR, Minor: $MINOR, Patch: $PATCH"
fi

# Split string into array
VERSION="1.2.3"
IFS='.' read -ra PARTS <<< "$VERSION"
echo "Major: ${PARTS[0]}"   # 1

# Trim whitespace (no built-in — use parameter expansion)
trim() {
    local str="$1"
    str="${str#"${str%%[![:space:]]*}"}"   # Remove leading whitespace
    str="${str%"${str##*[![:space:]]}"}"   # Remove trailing whitespace
    echo "$str"
}
trim "   hello world   "   # "hello world"
```

🔷 Short crisp interview answer

> "Bash has rich built-in string manipulation via parameter expansion — no external tools needed. `${var#pattern}` removes the shortest matching prefix, `${var##pattern}` removes the longest (I use `${path##*/}` as a fast basename). `${var%pattern}` removes shortest suffix, `${var%%pattern}` removes longest. `${var//old/new}` replaces all occurrences. These are faster than calling `sed` or `awk` for simple string operations because they avoid spawning a subprocess."

3.9 Exit Codes & Error Handling — `set -e`, `set -o pipefail`, `trap`

🔷 Why this matters more than anything else 🔥

A script without proper error handling is a **ticking time bomb in production**. It will silently continue after failures, leave systems in half-configured states, and cause incidents that are hard to debug.

🔷 Exit codes — the contract

```bash
# Every command returns an exit code 0-255
# 0     = success
# 1     = general error
# 2     = misuse of shell command
# 126   = command found but not executable
# 127   = command not found
# 128+N = killed by signal N (e.g., 130 = Ctrl+C = SIGINT)
# 255   = exit code out of range

# Check exit code
command
echo $?         # Print exit code

# Conditional on exit code
if command; then
    echo "Success"
else
    echo "Failed with code $?"
fi

# Negate exit code
if ! command; then
    echo "Command failed"
fi
```

🔷 `set -e` — exit on error

```bash
# Without set -e (default):
#!/usr/bin/env bash
cp important_file /nonexistent/   # Fails silently
rm -rf /production/data/           # CONTINUES even after failure!

# With set -e:
#!/usr/bin/env bash
set -e
cp important_file /nonexistent/   # Script EXITS HERE
rm -rf /production/data/           # Never reached

# Commands in if/while conditions don't trigger exit
if grep "pattern" file; then ...   # grep failure won't cause exit

# Commands with || don't trigger exit
command || true                    # Explicitly handle failure
command || echo "failed, continuing"
```

🔷 `set -o pipefail` — catch pipeline failures ⚠️

```
# Without pipefail:
set -e
cat nonexistent_file | grep "pattern" | wc -l
# cat fails (exit 1), but pipeline exit = wc's exit = 0
# Script does NOT exit!

# With pipefail:
set -eo pipefail
cat nonexistent_file | grep "pattern" | wc -l
# Now: pipeline fails if ANY command in it fails
# Script exits because cat failed

# Temporary pipefail disable
set +o pipefail
grep "optional_pattern" file | head -5  # grep might return 1 if no match
set -o pipefail

# The pipe silent failure pattern — most common CI/CD bug:
cat file | python3 process.py   # cat failure is hidden!
python3 process.py < file       # safer — no pipe, no hiding
# Always prefer input redirection over pipes when the left-hand command's
failure matters.
```

🔷 `set -u` — treat unset variables as errors

```
set -u
echo $UNDEFINED_VAR    # bash: UNDEFINED_VAR: unbound variable — exits!

# Safe access with default
echo "${UNDEFINED_VAR:-}"          # Empty string — safe
echo "${UNDEFINED_VAR:-default}"   # "default" — safe

# The golden standard — ALL THREE together:
set -euo pipefail
```

🔷 `trap` — catch signals and errors 🔥

```
# trap syntax:
# trap 'commands' SIGNAL [SIGNAL...]

# Signals you care about:
# EXIT    — runs when script exits (for any reason)
# ERR     — runs when command fails (respects set -e)
# INT     — Ctrl+C (SIGINT)
# TERM    — kill command (SIGTERM)
# HUP     — terminal hangup (SIGHUP)
```

```bash
# ── Pattern 1: Cleanup on exit ──────────────────────────────────
TMPDIR=$(mktemp -d)
trap "rm -rf '$TMPDIR'" EXIT     # Always cleanup temp files

# ── Pattern 2: Error handler with line number ───────────────────
error_handler() {
    local exit_code=$?
    local line_number=$1
    echo "ERROR: Script failed at line $line_number with exit code
$exit_code" >&2
}
trap 'error_handler $LINENO' ERR

# ── Pattern 3: Full production trap ─────────────────────────────
TMPFILE=""
LOCKFILE="/var/run/deploy.lock"

cleanup() {
    local exit_code=$?
    [[ -f "$TMPFILE" ]] && rm -f "$TMPFILE"
    [[ -f "$LOCKFILE" ]] && rm -f "$LOCKFILE"
    if [[ $exit_code -ne 0 ]]; then
        echo "Script failed with exit code: $exit_code" >&2
    fi
}
trap cleanup EXIT

# ── Pattern 4: Graceful shutdown ────────────────────────────────
KEEP_RUNNING=true
trap 'KEEP_RUNNING=false' INT TERM

while $KEEP_RUNNING; do
    process_next_job
    sleep 1
done
echo "Gracefully stopped"

# ── Pattern 5: Multiple traps ───────────────────────────────────
trap 'echo "Caught INT"; exit 130' INT
trap 'echo "Caught TERM"; exit 143' TERM
trap 'cleanup_function' EXIT

# ── Disable a trap ──────────────────────────────────────────────
trap - EXIT    # Reset EXIT trap to default
```

🔷 Complete error handling template

```bash
#!/usr/bin/env bash
set -euo pipefail
```

```
IFS=$'\n\t'

# Error context
err_report() {
    echo "Error on line $(caller)" >&2
}
trap err_report ERR

# Cleanup
TMPDIR=""
cleanup() {
    [[ -n "$TMPDIR" && -d "$TMPDIR" ]] && rm -rf "$TMPDIR"
}
trap cleanup EXIT

# Main
TMPDIR=$(mktemp -d)
# ... rest of script
```

🔷 Short crisp interview answer

> "`set -euo pipefail` is the error handling trinity every production script should start with: `-e` exits on command failure, `-u` exits on undefined variable access, and `-o pipefail` makes pipelines fail if any stage fails. `trap` lets you catch signals and the special `EXIT` and `ERR` pseudo-signals — I always trap `EXIT` to clean up temp files and lock files regardless of how the script ends. The most critical production pattern is storing temp file paths in variables and trapping `EXIT` to remove them."

⚠ Tricky gotchas

```
# GOTCHA 1: set -e doesn't always behave as expected in subshells
set -e
(false; echo "still runs in subshell")

# GOTCHA 2: set -e interacts with functions used in conditionals
set -e
check() { grep "pattern" file; }  # grep returns 1 if no match
if check; then ...   # Correct — if absorbs the non-zero exit
check               # ❌ If grep finds nothing, script exits!

# GOTCHA 3: IFS=$'\n\t' — what does it do?
IFS=$'\n\t'    # Sets word splitting to ONLY newlines and tabs (not
spaces)
               # Prevents accidental splitting on spaces in filenames
               # Set together with set -euo pipefail at top of every
script

# GOTCHA 4: trap ERR doesn't fire in all subshell contexts
set -e
```

```
trap 'echo error' ERR
(false)    # Subshell exits, parent trap behavior varies by bash version
```

3.10 Process & Command Substitution — `$()`, `<()`, `>()`

🔷 Command substitution — `$()`

```
# Run command, capture its stdout as a string
TODAY=$(date +%Y-%m-%d)
echo "Today is $TODAY"

# Nested command substitution
OLDEST_FILE=$(ls -t $(find /var/log -name "*.log") | tail -1)

# In assignments, conditionals, arguments
if [[ $(wc -l < file.txt) -gt 100 ]]; then
    echo "File has more than 100 lines"
fi

DISK_USAGE=$(df / | awk 'NR==2 {print $5}' | tr -d '%')

# Old-style backticks — avoid!
TODAY=`date`    # ❌ Avoid — hard to nest, confusing
TODAY=$(date)   # ✅ Preferred

# Key point: $() runs in a SUBSHELL
# Variables set inside don't affect parent:
RESULT=$(MY_VAR="hello"; echo $MY_VAR)
echo $MY_VAR    # Empty — subshell can't modify parent
```

🔷 Process substitution — `<()` and `>()`

This is one of the most **powerful and underused** Bash features. It lets you treat the **output of a command as a file**.

```
# <(command) — command's stdout becomes a readable file-like descriptor
# >(command) — writes to command's stdin as if writing to a file

# Problem: diff needs two FILES, but you want to diff command outputs
# Without process substitution:
cmd1 > /tmp/out1
cmd2 > /tmp/out2
diff /tmp/out1 /tmp/out2
rm /tmp/out1 /tmp/out2

# With process substitution — elegant!
```

```
diff <(cmd1) <(cmd2)

# Real examples:
diff <(sort file1.txt) <(sort file2.txt)

# Compare remote file to local
diff <(ssh server "cat /etc/nginx/nginx.conf") /etc/nginx/nginx.conf

# Diff two AWS S3 bucket listings
diff <(aws s3 ls s3://bucket-a) <(aws s3 ls s3://bucket-b)

# Process substitution FIXES the pipe subshell problem!
# WRONG — variables lost:
counter=0
cat file | while read line; do
    (( counter++ ))
done
echo $counter    # Always 0!

# CORRECT — while loop stays in current shell:
counter=0
while IFS= read -r line; do
    (( counter++ ))
done < <(cat file)
echo $counter    # Correct!

# comm — compare sorted files
comm -23 <(sort list1) <(sort list2)   # Lines only in list1

# >() — write to process substitution
# tee to multiple destinations simultaneously
tee >(gzip > output.gz) >(wc -l) > output.txt

# Log to BOTH stdout AND a log file from entire script
exec > >(tee -a /var/log/script.log) 2>&1
```

🔷 Short crisp interview answer

> "`$()` is command substitution — it runs a command in a subshell and returns its stdout as a string. Process substitution `<(cmd)` and `>(cmd)` create named pipes that look like files — the key insight is that `<(cmd)` lets you pass command output where a filename is expected. I use it constantly to avoid temp files: `diff <(sort a) <(sort b)` or to fix the pipe-subshell variable scope problem by rewriting `cat file | while read` as `while read; done < <(cat file)` so the loop stays in the current shell."

3.11 Regex in Bash — `=~`, Character Classes, Anchors

🔷 Bash regex with `=~`

```bash
# =~ operator — only works inside [[ ]]
# Match result stored in BASH_REMATCH array

# Basic match
if [[ "hello123" =~ [0-9]+ ]]; then
    echo "Contains digits"
fi

# Capture groups → BASH_REMATCH
VERSION="v2.14.3"
if [[ "$VERSION" =~ ^v([0-9]+)\.([0-9]+)\.([0-9]+)$ ]]; then
    echo "Full match:  ${BASH_REMATCH[0]}"   # v2.14.3
    echo "Major:       ${BASH_REMATCH[1]}"   # 2
    echo "Minor:       ${BASH_REMATCH[2]}"   # 14
    echo "Patch:       ${BASH_REMATCH[3]}"   # 3
fi

# Validate IP address
IP="192.168.1.256"
OCTET="([0-9]|[1-9][0-9]|1[0-9]{2}|2[0-4][0-9]|25[0-5])"
if [[ "$IP" =~ ^${OCTET}\.${OCTET}\.${OCTET}\.${OCTET}$ ]]; then
    echo "Valid IP"
else
    echo "Invalid IP"
fi

# Validate email (simplified)
if [[ "$EMAIL" =~ ^[a-zA-Z0-9._%+-]+@[a-zA-Z0-9.-]+\.[a-zA-Z]{2,}$ ]];
then
    echo "Valid email"
fi

# ⚠ Store regex in variable for complex patterns — do NOT quote it
REGEX='^[0-9]{4}-[0-9]{2}-[0-9]{2}$'
if [[ "$DATE" =~ $REGEX ]]; then    # No quotes around $REGEX!
    echo "Valid date"
fi
# Quoting the regex literal disables special chars on some bash versions:
if [[ "$DATE" =~ '^[0-9]{4}' ]]; then    # ❌ ^ treated as literal
if [[ "$DATE" =~ ^[0-9]{4} ]]; then      # ✅ Unquoted regex
```

🔷 Character classes

```bash
# POSIX classes (portable, work inside [ ])
[:alpha:]    # Letters a-z A-Z
[:digit:]    # Digits 0-9
[:alnum:]    # Letters + digits
[:space:]    # Whitespace (space, tab, newline)
[:upper:]    # Uppercase A-Z
```

```
[:lower:]    # Lowercase a-z
[:print:]    # Printable characters
[:punct:]    # Punctuation

# Used in regex:
[[ "$str" =~ ^[[:alpha:]]+$ ]]    # All letters
[[ "$str" =~ ^[[:alnum:]_]+$ ]]   # Letters, digits, underscore

# Anchors
^     # Start of string
$     # End of string
```

🔷 Short crisp interview answer

> "Bash supports ERE regex via the `=~` operator inside `[[ ]]`. Capture groups are stored in `BASH_REMATCH` — index 0 is the full match, 1+ are groups. The key gotcha is to store complex regexes in a variable and use it unquoted: `[[ $var =~ $REGEX ]]` — quoting the regex prevents special characters from being interpreted. I use it most for input validation: version strings, IP addresses, dates."

3.12 `xargs` & Parallel Execution

🔷 `xargs` — build and execute commands from stdin

```
# Basic: pipe list → xargs → command
find /var/log -name "*.log" | xargs wc -l

# Problem without xargs — too many arguments:
ls *.log | rm          # ❌ rm doesn't read stdin
find . -name "*.tmp" -exec rm {} \;   # Works but spawns rm per file

# xargs solution — batches arguments
find . -name "*.tmp" | xargs rm       # One rm call with many args

# -I: replace string (for positional arg control)
find . -name "*.txt" | xargs -I{} cp {} /backup/

# Handling filenames with spaces — CRITICAL
find . -name "*.log" -print0 | xargs -0 wc -l
# -print0 uses null byte delimiter, -0 reads null-delimited
# This safely handles filenames with spaces, newlines, special chars

# -n: arguments per command invocation
echo "1 2 3 4 5 6" | xargs -n2 echo
# echo 1 2
# echo 3 4
# echo 5 6
```

```
# -P: parallel execution
find . -name "*.gz" | xargs -P4 -I{} gunzip {}
# Runs up to 4 gunzip processes simultaneously

# With ssh — deploy to multiple servers in parallel
cat servers.txt | xargs -P10 -I{} ssh {} "systemctl restart app"

# Combine -n and -P
find /data -name "*.csv" | xargs -P8 -n1 python3 process.py

# -t: print command before executing (debug mode)
echo "file1 file2 file3" | xargs -t -n1 wc -l

# Pass extra arguments with xargs
find . -name "*.sh" | xargs -I{} bash -c 'echo "Checking: {}"; bash -n "
{}"'
```

🔷 Parallel execution patterns

```
# Pattern 1: Background jobs + wait
SERVERS=(web01 web02 web03 web04 web05)
for server in "${SERVERS[@]}"; do
    ssh "$server" "deploy.sh" &    # & = background
done
wait    # Wait for ALL background jobs

# Pattern 2: Capture exit codes from parallel jobs
PIDS=()
for server in "${SERVERS[@]}"; do
    ssh "$server" "deploy.sh" &
    PIDS+=($!)    # $! = PID of last background process
done

FAILED=0
for i in "${!PIDS[@]}"; do
    if ! wait "${PIDS[$i]}"; then
        echo "FAILED: ${SERVERS[$i]}" >&2
        FAILED=1
    fi
done
exit $FAILED

# Pattern 3: GNU parallel (when installed)
cat servers.txt | parallel -j10 ssh {} "systemctl restart app"
parallel -j4 gzip ::: /var/log/*.log
parallel -j8 python3 process.py ::: data/*.csv

# Pattern 4: Semaphore pattern (limit concurrency in bash)
MAX_JOBS=4
CURRENT_JOBS=0
```

```
for item in "${ITEMS[@]}"; do
    process_item "$item" &
    (( CURRENT_JOBS++ ))
    if (( CURRENT_JOBS >= MAX_JOBS )); then
        wait -n 2>/dev/null || wait    # Wait for any one job to finish
        (( CURRENT_JOBS-- ))
    fi
done
wait    # Wait for remaining jobs
```

🔷 Short crisp interview answer

> "`xargs` reads items from stdin and passes them as arguments to a command — it's the bridge between pipes and commands that don't read stdin. The most important flags are `-I{}` for positional control, `-P` for parallelism, and always `-print0 | xargs -0` when filenames might have spaces. For parallel deployments in scripts I use background jobs with `&`, collect PIDs into an array, then check each one with `wait $pid` to detect failures."

⚠ Tricky gotchas

```
# GOTCHA 1: xargs with no input runs the command with no args
echo "" | xargs rm     # Runs: rm  — with no args, does nothing (ok)
# But:
echo "" | xargs rm -rf # This may run rm -rf with no path — dangerous!
# Fix: use -r (--no-run-if-empty) to skip if input is empty
echo "" | xargs -r rm

# GOTCHA 2: Quote handling with -I{}
find . -name "*.txt" | xargs -I{} echo "File: {}"
# If filename has spaces: File: my file.txt — works with -I{}
# But without -I{}:
find . -name "*.txt" | xargs echo  # Spaces break filenames!

# GOTCHA 3: wait with no args waits for ALL background jobs
# wait $pid waits for a SPECIFIC job and returns its exit code
for pid in "${PIDS[@]}"; do
    wait "$pid"     # Returns the exit code of that specific job
    echo "Job $pid exited with $?"
done
```

3.13 File Descriptor Manipulation

🔷 What file descriptors are

```
Every process has a table of open files:

┌────┬──────────────────────────────┐
│ FD │ Description                  │
├────┼──────────────────────────────┤
│ 0  │ stdin  (keyboard by default) │
│ 1  │ stdout (terminal by default) │
│ 2  │ stderr (terminal by default) │
│ 3+ │ available for scripts to use │
└────┴──────────────────────────────┘
```

🔷 Custom file descriptors with `exec`

```bash
# Open custom file descriptors with exec
exec 3>/tmp/debug.log          # FD 3 = write to file
exec 4</etc/hosts              # FD 4 = read from file
exec 5>&1                      # FD 5 = copy of stdout (save it)

# Write to FD 3
echo "debug message" >&3

# Read from FD 4
read -u 4 line                 # -u 4 reads from FD 4
echo "Read: $line"

# Close file descriptors
exec 3>&-                      # Close FD 3 (write)
exec 4<&-                      # Close FD 4 (read)

# Redirect entire script's output to a log
exec >> /var/log/myscript.log 2>&1
echo "This goes to log file"   # stdout + stderr → log file

# Save and restore stdout
exec 5>&1          # Save stdout to FD 5
exec > /tmp/out    # Redirect stdout to file
echo "to file"
exec >&5           # Restore stdout from FD 5
exec 5>&-          # Close FD 5
echo "to terminal" # Back to terminal

# Logging to BOTH file and terminal using tee via FD
exec > >(tee -a /var/log/script.log) 2>&1

# Read two files simultaneously with different FDs
exec 3</path/to/file1
exec 4</path/to/file2
while true; do
    IFS= read -r -u3 line1 || break
    IFS= read -r -u4 line2 || break
```

```
    echo "File1: $line1 | File2: $line2"
done
exec 3<&-
exec 4<&-
```

🔷 Short crisp interview answer

> "File descriptors are integer handles to open files or streams. Every process starts with 0 (stdin), 1 (stdout), 2 (stderr). With `exec` I can open custom FDs 3 and above — useful for reading two files simultaneously in a loop, or saving and restoring stdout around a section that redirects output. The pattern `exec > >(tee -a logfile) 2>&1` at the top of a script sends all output to both terminal and log file without wrapping every command."

3.14 Subshells vs Child Processes — `( )` vs `{ }`

🔷 The critical scoping distinction ⚠️🔥

```
PARENT SHELL
│
├── () — Subshell
│      ├── Copies parent's variables/environment
│      ├── Changes DO NOT propagate to parent
│      └── Runs in separate process (fork)
│
└── {} — Group command (current shell)
       ├── Runs in SAME process
       ├── Changes DO affect parent
       └── No fork — more efficient
```

```
# Subshell ( ) — changes are LOCAL
X=10
(
    X=99
    echo "Inside subshell: $X"   # 99
)
echo "After subshell: $X"        # 10 — unchanged!

# Group { } — changes persist (note: space after { and ; before })
X=10
{
    X=99
    echo "Inside group: $X"      # 99
}
echo "After group: $X"           # 99 — CHANGED!
```

🔷 When to use each

```
# Use () — subshell — when you want ISOLATION

# cd without affecting parent
(cd /tmp && ls)
echo "$PWD"        # Still original directory

# Run something in background
(long_running_process) &

# Contain a set of commands with their own variables
(
    set -x          # Debug only this section
    risky_command
)

# Use {} — group — when you want SHARED SCOPE

# Group redirections efficiently (no fork)
{ cmd1; cmd2; cmd3; } > output.txt

# Error handling groups
{ command1 && command2 && command3; } || die "Pipeline failed"

# When efficiency matters (no fork per iteration)
for i in {1..1000}; do
    { process "$i"; }
done
```

🔷 How pipes relate

```
# Every pipeline segment runs in a subshell
echo "hello" | read VAR    # read runs in subshell — VAR lost!
echo $VAR                  # Empty

# Each pipe segment is a separate subshell
A=1; B=2
echo "test" | { read line; A=10; B=20; echo "$line $A $B"; }
echo "$A $B"    # Still "1 2" — pipe group is a subshell!

# Fix with process substitution to keep current shell
while IFS= read -r line; do
    process "$line"
done < <(generate_data)    # while is in current shell
```

🔷 Short crisp interview answer

> "`()` creates a subshell — a forked copy of the current shell where variable changes are isolated and don't propagate back to the parent. `{}` groups commands in the current shell — changes persist and no fork happens, making it more efficient. I use subshells when I need isolation like temporary `cd` without affecting the parent, and command groups when I want to redirect a series of commands to a single file or handle errors as a unit."

⚠ Tricky gotchas

```
# GOTCHA 1: {} syntax is strict
{echo "hello"}     # ❌ No spaces — command not found
{ echo "hello"; }  # ✅ Space after {, semicolon before }

# GOTCHA 2: Pipes always create subshells — even with {}
echo "test" | { read line; MY_VAR="set"; }
echo $MY_VAR    # Empty! Pipe group is still a subshell

# GOTCHA 3: $$ vs $BASHPID in subshells
echo "Parent PID: $$"
(echo "Subshell $$")       # SAME as parent — $$ doesn't change in
subshells!
(echo "Subshell $BASHPID") # Different — $BASHPID reflects actual subshell
PID

# GOTCHA 4: set -e in subshells
set -e
(false)    # Subshell exits with 1 — does the parent exit?
           # In bash: yes, the parent will exit because () returned non-
zero
```

3.15 Script Hardening — `set -euo pipefail`, Defensive Patterns

🔷 The complete hardening header

```
#!/usr/bin/env bash
# ─── Strict mode ─────────────────────────────────────────────────────
set -euo pipefail
IFS=$'\n\t'

# -e          : exit on error
# -u          : exit on undefined variable
# -o pipefail : fail on any pipe stage failure
# IFS=$'\n\t' : prevent word splitting on spaces
```

🔷 Defensive coding patterns

```
# 1. Validate required arguments
usage() { echo "Usage: $0 <env> <version>" >&2; exit 1; }
[[ $# -lt 2 ]] && usage

# 2. Validate argument values
[[ "$1" =~ ^(production|staging|dev)$ ]] || { echo "Invalid env: $1" >&2;
exit 1; }

# 3. Require commands to exist before starting
for cmd in jq curl aws kubectl; do
    command -v "$cmd" &>/dev/null || { echo "Required: $cmd not found";
exit 1; }
done

# 4. Validate files before using
[[ -f "$CONFIG_FILE" ]] || { echo "Config not found: $CONFIG_FILE"; exit
1; }
[[ -r "$CONFIG_FILE" ]] || { echo "Config not readable: $CONFIG_FILE";
exit 1; }

# 5. Use readonly for constants
readonly MAX_RETRIES=5
readonly TIMEOUT=30
readonly SCRIPT_DIR="$(cd "$(dirname "${BASH_SOURCE[0]}")" && pwd)"

# 6. Always quote variables
cp "$source" "$destination"       # ✅
cp $source $destination           # ❌

# 7. Use ${var:-default} for optional vars
LOG_LEVEL="${LOG_LEVEL:-INFO}"
TIMEOUT="${TIMEOUT:-30}"

# 8. Prevent running as root (or require it)
[[ $EUID -ne 0 ]] && { echo "Must run as root"; exit 1; }
[[ $EUID -eq 0 ]] && { echo "Don't run as root!"; exit 1; }

# 9. Locking — prevent concurrent runs
LOCKFILE="/var/run/${SCRIPT_NAME}.lock"
exec 9>"$LOCKFILE"
flock -n 9 || { echo "Script already running (lockfile: $LOCKFILE)"; exit
1; }

# 10. Temp file safety
TMPFILE=$(mktemp)
trap "rm -f '$TMPFILE'" EXIT

# 11. Validate numeric inputs
is_integer() { [[ "$1" =~ ^-?[0-9]+$ ]]; }
is_integer "$REPLICAS" || { echo "REPLICAS must be integer, got:
```

```
$REPLICAS"; exit 1; }

# 12. Confirm destructive actions in production
if [[ "$ENVIRONMENT" == "production" ]]; then
    read -r -p "Deploying to PRODUCTION. Are you sure? [yes/no]: " CONFIRM
    [[ "$CONFIRM" == "yes" ]] || { echo "Aborted"; exit 0; }
fi

# 13. Dry-run mode
DRY_RUN="${DRY_RUN:-false}"
run() {
    if [[ "$DRY_RUN" == "true" ]]; then
        echo "[DRY-RUN] $*"
    else
        "$@"
    fi
}
run systemctl restart app

# 14. Log all commands in debug mode
[[ "${DEBUG:-false}" == "true" ]] && set -x

# 15. CI/CD deployment script checklist:
# set -euo pipefail + IFS=$'\n\t' at top
# Validate all required env vars with :"${VAR:?message}"
# flock for deploy locking
# trap 'cleanup' EXIT for guaranteed cleanup
# All error messages to >&2
```

🔷 Short crisp interview answer

> "Script hardening starts with `set -euo pipefail` and `IFS=$'\\n\\t'` at the top. Beyond that: validate all arguments and their values before using them, use `readonly` for constants, always quote variables, use `${var:-default}` for optional parameters, check required commands exist before running, use `trap EXIT` for cleanup, and use `flock` for locking to prevent concurrent runs. For destructive production scripts I add explicit confirmation prompts and dry-run modes."

3.16 Performance in Bash — When to Use Bash vs Python

🔷 Bash performance profile

```
Fast in Bash:
✅ Running external commands and pipelines
✅ File I/O via redirection (streaming — no memory bottleneck)
✅ Simple string operations via parameter expansion
✅ Process management, signals, daemons
✅ Orchestrating other tools
```

```
Slow in Bash:
❌ String processing in pure-bash loops
❌ Math — especially floating point
❌ Large data structures
❌ Complex data parsing (JSON, YAML, XML)
❌ HTTP/API calls in loops
❌ Anything requiring more than ~200 lines of logic
```

🔷 The fork cost — hidden performance killer

```bash
# EVERY $() spawns a subshell (fork + exec)
# In a tight loop, this is catastrophic

# Slow — 1000 forks for bc + 1000 forks for echo!
for i in $(seq 1 1000); do
    result=$(echo "$i * 2" | bc)
done

# Fast — bash built-in arithmetic, zero forks
for (( i=1; i<=1000; i++ )); do
    result=$(( i * 2 ))
done

# Slow — fork per line of file
while read line; do
    COUNT=$(echo "$line" | wc -w)  # Fork per line!
done < file

# Fast — one awk process handles entire file
awk '{print NF}' file

# Slow — calling date inside loop
for server in "${SERVERS[@]}"; do
    LOG="$(date +%Y%m%d)-${server}.log"   # Fork per iteration
done

# Fast — call once, reuse
TODAY=$(date +%Y%m%d)
for server in "${SERVERS[@]}"; do
    LOG="${TODAY}-${server}.log"    # Zero forks in loop
done

# Slow — string manipulation with external tools
for word in "${WORDS[@]}"; do
    UPPER=$(echo "$word" | tr '[:lower:]' '[:upper:]')  # Fork!
done

# Fast — bash built-in case conversion (bash 4.0+)
for word in "${WORDS[@]}"; do
```

```
    UPPER="${word^^}"    # Zero forks
done
```

🔷 When to switch to Python

```
Switch to Python when:
├── Script exceeds ~200 lines of logic
├── You need JSON/YAML/XML parsing (use jq for simple JSON, Python for
complex)
├── You need floating point math
├── You need complex data structures (dicts of dicts, objects)
├── You need proper error handling with context/stack traces
├── You need to make multiple HTTP API calls
├── You need unit tests
├── You need cross-platform compatibility (Windows too)
└── You need to reuse logic across multiple scripts (modules)

Stay in Bash when:
├── Script is orchestrating other tools
├── Script is < 200 lines
├── Main work is file I/O, pipelines, process management
├── Script lives in a minimal container with no Python
└── Speed of development matters more than code quality
```

🔷 The hybrid pattern — Bash as glue, Python for logic

```
#!/usr/bin/env bash
set -euo pipefail

# Bash handles: argument parsing, orchestration, cleanup
ENVIRONMENT="$1"
TIMESTAMP=$(date +%Y%m%d_%H%M%S)
TMPFILE=$(mktemp)
trap "rm -f '$TMPFILE'" EXIT

# Python handles: business logic, JSON, math
python3 << EOF
import json, sys

config = json.load(open('/etc/app/config.json'))
servers = [s for s in config['servers'] if s['env'] == '${ENVIRONMENT}']

for s in servers:
    print(s['hostname'])
EOF

# Bash resumes: act on Python's output
```

```
while IFS= read -r server; do
    echo "Deploying to: $server"
    ssh "$server" "./deploy.sh"
done < <(python3 get_servers.py "$ENVIRONMENT")
```

🔷 Performance benchmarking tricks

```
# Time a command
time ./script.sh

# Time a section
START=$(date +%s%N)    # Nanoseconds
do_work
END=$(date +%s%N)
echo "Elapsed: $(( (END - START) / 1000000 ))ms"

# Count forks (subprocesses spawned)
strace -f -e trace=clone,fork,vfork ./script.sh 2>&1 | grep -c
"clone\|fork"

# Profile with bash debug timing
PS4='+ $(date "+%s.%N") '
bash -x script.sh 2>timing.log
```

🔷 Short crisp interview answer

> "The key performance insight is that every `$()` is a fork — spawning a subshell has overhead. In loops this compounds: calling `date` or `wc` inside a 1000-iteration loop is 1000 forks. The fix is to hoist expensive calls outside loops and use bash built-ins like `$(( ))` for math and parameter expansion for string ops — no subprocess spawned. I switch to Python when the script exceeds ~200 lines, needs JSON parsing, floating point, HTTP calls, or unit tests. Bash is excellent as an orchestrator but poor as a data processor."

🏆 Category 3 — Complete Mental Model

```
BASH SCRIPTING DECISION TREE
═══════════════════════════════

Starting a script?
    └── #!/usr/bin/env bash + set -euo pipefail + IFS=$'\n\t'

Need to store data?
    ├── Single value      → variable with "double quotes"
```

```
    ├── List of items     → indexed array "${arr[@]}"
    └── Key-value pairs   → declare -A associative array

Need to process text?
    ├── Simple string ops → parameter expansion ${var//old/new}
    ├── Column data       → awk
    └── Substitution      → sed

Need to make decisions?
    ├── String/file tests → [[ ]] (always prefer over [ ])
    ├── Arithmetic        → (( ))
    └── Multiple values   → case statement

Need to repeat?
    ├── Known list        → for item in "${array[@]}"
    ├── File/stream       → while IFS= read -r line
    └── Condition-based   → while [[ condition ]]

Need to handle errors?
    ├── Exit codes        → $? captured IMMEDIATELY
    ├── Cleanup           → trap 'cleanup' EXIT
    └── Signals           → trap 'handler' INT TERM

Need parallelism?
    ├── Simple            → cmd & ; wait
    ├── Track failures    → pids=($!) ; wait "${pids[@]}"
    └── Many items        → xargs -P N

Script getting complex?
    ├── < 200 lines       → Bash with functions
    └── > 200 lines or needs JSON/math/tests → Python
```

⚠ Master Gotcha List

#	Gotcha	Fix
1	`NAME = "val"` — space around `=`	`NAME="val"` — no spaces ever
2	`$@` vs `$*` with spaces in args	Always use `"$@"` to preserve argument boundaries
3	`[ ]` word-splits unquoted variables	Use `[[ ]]` or always quote: `[ "$var" ]`
4	`set -e` doesn't catch all failures	Add `-o pipefail` and `trap ERR`
5	Variables in pipe subshells are lost	Use `< <(cmd)` process substitution instead
6	Missing `local` in functions	Always `local` every variable inside functions
7	`return` vs `exit` in functions	`return` exits function only; `exit` exits entire script
8	`$?` reset by every command	Capture with `rc=$?` immediately after the command
9	`unset arr[0]` makes sparse array	Rebuild array or account for gaps in index iteration

#	Gotcha	Fix
10	`()` vs `{}` scoping	`()` = subshell (isolated), `{}` = current shell (shared)
11	Regex in `[[ =~ ]]` — don't quote regex	Store in variable or leave unquoted in the expression
12	`for f in $(find ...)` breaks on spaces	Use `while IFS= read -r` with `find -print0 | xargs -0`
13	`echo -e` is not portable	Use `printf` for escape sequences in scripts
14	Glob expands to literal if no match	Use `shopt -s nullglob` or check `[[ -f "$f" ]]`
15	`$ARRAY` only gets first element	Use `"${ARRAY[@]}"` for all elements

🔥 Top Interview Questions

Q1: What does `set -euo pipefail` do and why is each flag important?

> `-e` exits the script if any command fails. `-u` treats unset variables as errors instead of silently using empty string — this is critical because `rm -rf "$UNSET_VAR/"` becomes `rm -rf "/"`. `-o pipefail` makes a pipeline fail if any command in it fails, not just the last one. Together they turn bash from a silent-failure language into one that blows up loudly on mistakes — exactly what you want in production automation.

Q2: What's the difference between `$@` and `$*`?

> Both expand to all positional parameters, but `"$@"` expands to separate quoted words — each argument is preserved as its own word even if it contains spaces. `"$*"` joins all arguments into a single string using the first character of IFS as separator. Always use `"$@"` when forwarding arguments to preserve original argument boundaries.

Q3: How do you safely read a file line by line in Bash?

```
while IFS= read -r line; do
    echo "$line"
done < file.txt
# IFS= prevents stripping leading/trailing whitespace
# -r prevents backslash interpretation (\n stays as \n)
```

Q4: Why can't you set a variable inside a while loop reading from a pipe?

> Because the pipe creates a subshell for each pipeline segment, and subshells can't modify the parent shell's variables. Fix by using process substitution `< <(command)` which keeps the while loop in the current shell:

```
# Wrong — TOTAL always 0:
cat numbers.txt | while read -r n; do (( TOTAL += n )); done

# Correct — TOTAL accumulates:
while read -r n; do (( TOTAL += n )); done < <(cat numbers.txt)
```

Q5: How do you trap and clean up temp files?

```
TMPFILE=$(mktemp)
trap "rm -f '$TMPFILE'" EXIT
# EXIT fires on any exit: normal completion, set -e error, or signal
```

Q6: What's the difference between `[ ]` and `[[ ]]`?

> `[ ]` is the `test` command — POSIX portable, but treats its arguments as shell words, so unquoted variables get word-split and `<`/`>` need escaping. `[[ ]]` is a bash keyword — no word splitting occurs, `&&`/`||` work directly, `<`/`>` compare strings without escaping, and `=~` enables regex matching. Always prefer `[[ ]]` in bash scripts unless writing POSIX sh.

Q7: How do you run tasks in parallel and wait for all to complete?

```
pids=()
for item in "${items[@]}"; do
    process "$item" &
    pids+=($!)
done

failed=0
for pid in "${pids[@]}"; do
    wait "$pid" || { echo "Job $pid failed"; failed=1; }
done
exit $failed
```

Q8: What is process substitution and when would you use it?

> `<(cmd)` creates a named pipe (FIFO) that looks like a temporary file containing the output of `cmd`. I use it in two key situations: when a command requires file arguments but I have command output — `diff <(sort a.txt) <(sort b.txt)` — and to fix the pipe-subshell variable scope problem: `while read line; done < <(generate_data)` keeps the while loop in the current shell so variable changes persist.

Q9: How does `local` affect variable scoping in Bash functions?

> Without `local`, any variable set inside a function modifies the global scope — it can silently overwrite a variable with the same name in the calling scope. `local` restricts the variable to the function's scope and any functions it calls. Always use `local` for every variable inside a function. Forgetting this is a common source of subtle bugs where a function accidentally corrupts outer state.

Q10: How would you write a retry loop with exponential backoff?

```
retry_with_backoff() {
    local max_attempts=$1
    local delay=1
    shift
    local attempt=0

    while (( attempt < max_attempts )); do
        if "$@"; then
            return 0
        fi
        attempt=$(( attempt + 1 ))
        if (( attempt < max_attempts )); then
            echo "Attempt $attempt failed. Retrying in ${delay}s..." >&2
            sleep "$delay"
            delay=$(( delay * 2 ))    # Exponential backoff
        fi
    done
    echo "All $max_attempts attempts failed" >&2
    return 1
}

# Usage:
retry_with_backoff 5 curl -sf https://api.example.com/health
retry_with_backoff 3 kubectl apply -f deployment.yaml
```

Document covers all 16 topics in Category 3: Bash Scripting — from shebang and script structure through advanced error handling, parallel execution, and performance optimization. Every section includes real production patterns, interview-ready answers, and critical gotchas.